

Algorithmes de tris

Pagès Stanislas, Jorel Raphaël,
Université de Perpignan Via Domitia

Table des matières

1	Généralités	2
1.1	Introduction	2
1.2	Caractéristiques	2
1.2.1	Caractères	2
1.2.2	Complexités	2
1.3	Conventions	3
2	Principes de quelques tris	4
2.1	Tris par comparaisons	4
2.1.1	Tri à bulles (bubble sort)	4
2.1.2	Tri par sélection (selection sort)	5
2.1.3	Tri par insertion (insertion sort)	5
2.1.4	Tri rapide (quick sort)	7
2.1.5	Tri fusion (merge sort)	8
2.1.6	Tri par tas (heap sort)	9
2.2	Tris sans comparaisons	11
2.2.1	Tri par base (radix sort)	11
2.2.2	Tri comptage (counting sort)	12
2.2.3	Tri par paquets (bucket sort)	12
2.3	Réseaux de tris (sorting networks)	14
2.3.1	Généralités	14
2.3.2	Réseau de tri bitonique (bitonic sorter)	16
2.3.3	Réseau de tri pair - impair (odd-even merge sort)	18
3	Résultats expérimentaux	20
3.1	Tri par comparaisons	20
3.2	Tri par comparaisons rapides	21
3.3	Tri sans comparaisons	22
3.4	Interprétation	24
3.5	Conclusion	25
4	Algorithmes	26
4.1	Tri à bulles	26
4.2	Tri sélection	26
4.3	Tri insertion	27
4.4	Tri rapide	27
4.5	Tri fusion	28
4.6	Tri par tas	29
4.7	Tri par base	30
4.8	Tri comptage	30
4.9	Tri par paquets	31
4.10	Tri pair-impair	31
5	Codes sources	32

Chapitre 1

Généralités

“Imagine how hard it would be a dictionary if its words were not alphabetized!”
Donald E. Knuth, in The Art of Computer Programming [1]

1.1 Introduction

Les algorithmes de tris, une bien grande histoire... Le fait de ranger des objets n'est pas apparu avec l'informatique. De toute évidence, nos ancêtres les homo-sapiens devaient déjà avoir besoin de ranger, d'ordonner leurs biens afin de les retrouver. Quelque chose de bien humain, tout compte fait. L'exemple le plus concret est le dictionnaire, ou l'annuaire. Ces derniers seraient bien moins pratiques s'ils n'étaient pas organisés de façon logique (par lettre, dans ce cas). L'analogie à l'informatique est simple, des données rangées selon un certain ordre sont forcément plus simplement et rapidement retrouvables.

Cependant, malgré l'utilisation très importantes des tris dans les ordinateurs, il n'est pas aisé de trouver des informations purement historique sur les algorithmes de tri et donc ce document n'en aura que très peu. Comme le fait très bien remarquer D. E. Knuth, le terme **tri** n'est pas forcément le plus approprié dans ce qu'il fait réellement. Pour la plupart des gens, le terme tri évoque surement plus le fait de ranger par catégories, ou alors faire la différence entre ce qu'il faut garder et ce que l'on jette. D'autres termes seraient peut-être plus adéquat, mais c'est le terme de **tri** qui a été choisi par les informaticiens[1].

En résumé, un algorithme de tri est un algorithme qui permet de ranger les éléments d'une liste de valeurs dans un certains ordre. Les ordres les plus utilisés sont l'ordre numérique et l'ordre lexicographique.

1.2 Caractéristiques

Différentes caractéristiques sont utilisées pour comparer les algorithmes entre eux, permettant de mesurer l'efficacité de chacun d'eux. Nous vous les présentons donc ci-après car nous en reparlerons lorsque de nos explications plus précises sur quelques algorithmes.

1.2.1 Caractères

Il existe deux caractères importants étudiés pour chaque algorithme de tris, le caractère en place et le caractère stable.

En place Un algorithme est dit en place s'il n'utilise qu'un nombre limité de mémoire externe, et qu'il modifie directement la structure qu'il traite.

Stable Un algorithme de tri est dit stable si, lors de son exécution et donc de la mise en ordre des éléments, il conserve l'ordre d'apparition initial de ceux-ci.

1.2.2 Complexités

La complexité est une notion mathématique qui étudie la quantité de ressources (spatiale et temporelle) nécessaires à l'exécution d'un algorithme. Elle s'exprime en fonction du nombre d'entrées (n) de l'algorithme et

permet de mesurer une certaine efficacité de celui-ci. C'est donc un élément de comparaison important dans la classification des tris.

Temporelle Elle représente le coût en temps d'un algorithme et permet de savoir s'il est acceptable ou non à l'échelle humaine. On distingue trois complexités temporelles différentes, celle dans le pire cas (O), celle dans le meilleur (Ω) et enfin celle en moyenne (Θ).

Spatiale Cette complexité représente le coût mémoire de l'algorithme.

1.3 Conventions

Nous pensons qu'il est de bon ton d'établir quelques conventions, afin de mieux expliquer ce qui suit. Donc, par la suite, nous considérerons n **valeurs à ranger dans l'ordre croissant**, ces valeurs seront des **entiers** dans les exemples donnés, rangées dans un **tableau**, noté T . Un tableau est l'équivalent informatique du vecteur mathématique. Les données du tableau sont indexées par un indice (un nombre entier), nous écrivons $T[i]$ pour accéder à la valeur du tableau T d'indice i .

$$\vec{u} = (u_1, u_2, \dots, u_n)$$

$$T = \begin{array}{|c|c|c|c|} \hline T[1] & T[2] & \dots & T[n] \\ \hline \end{array}$$

Chapitre 2

Principes de quelques tris

Afin de couvrir un peu toutes les manières d'envisager un algorithme de tri, nous nous sommes efforcés de faire les tris qui nous paraissaient les plus intéressants.

2.1 Tris par comparaisons

2.1.1 Tri à bulles (bubble sort)

Le tri à bulles est un des premiers enseignés à ceux qui s'intéressent de près à l'informatique. Et pour cause, ce tri est relativement simple, le principe étant de repousser la valeur la plus importante vers un extremum du tableau à trier.

Principe

Pour ce faire, nous parcourons le tableau une première fois, et comparons les $T[j]$ avec leur suivant ($T[j+1]$), $\forall j \in \{1, \dots, n-1\}$, et les échangeons si $T[j] > T[j+1]$. A la fin du premier parcours, $T[n] = \max(T[1], \dots, T[n])$ et le tableau est trié de $T[n]$ à $T[1]$, puisqu'il n'y a qu'une seule valeur. Si nous faisons un second parcours, nous aurons $T[n-1] = \max(T[1], \dots, T[n-1])$, et le tableau sera trié de $T[n-1]$ à $T[n]$, puisque $T[n-1] \leq T[n]$. A chaque nouveau parcours, nous réduisons la taille du tableau à traiter de 1. Ainsi, à l'itération i , nous chercherons $T[n-i] = \max(T[1], \dots, T[n-i])$. Le tableau est trié de $T[n-i]$ à $T[n]$ si ce dernier l'est de $T[n-i+1]$ à $T[n]$. Le tableau étant trié à $i = 1$, un raisonnement par récurrence nous montre que le tableau sera trié à $i = n$.

Exemple

5	1	12	-5	16
5	1	12	-5	16
1	5	12	-5	16
1	5	12	-5	16
1	5	-5	12	16
1	5	-5	12	16
1	5	-5	12	16
1	5	-5	12	16
1	-5	5	12	16
1	-5	5	12	16
-5	1	5	12	16
-5	1	5	12	16

Caractères

Ce tri est en place, il modifie directement la structure qu'il utilise, il est également stable si l'algorithme ne définit pas d'échange de deux valeurs si elles sont égales. En effet, elles conserveront leur position relative.

Complexités

Temporelle Cet algorithme parcourt toujours les valeurs de la même manière. Comptons le nombre de comparaisons nécessaires pour n valeurs à trier, cela donnera un bon ordre d'idée de la complexité temporelle.

A chaque itération i , l'algorithme parcourt $n - i$ valeurs, la formule résultant est donc :

$$\sum_{i=0}^n (n - i) = 1 + 2 + \dots + n = \frac{n(n+1)}{2} = O(n^2)$$

Ce nombre de comparaison étant obligatoirement le même dans le meilleur et le pire cas, vu que l'algorithme procède toujours de cette manière, sa complexité temporelle est en $\Theta(n^2)$.

Spatiale La complexité spatiale est en $\Theta(1)$ car l'algorithme est stable, donc utilise très peu de mémoire externe.

2.1.2 Tri par sélection (selection sort)

Le tri par sélection est particulièrement simple et inefficace sur de grandes entrées, néanmoins dans le cadre de notre étude il est intéressant de le voir à titre comparatif.

Principe

Au départ, l'algorithme cherche la valeur minimale du tableau de $T[1]$ à $T[n]$, et la place en $T[1]$. Il cherche ensuite la valeur minimale de $T[2]$ à $T[n]$ pour la placer en $T[2]$, et ainsi de suite. Pour généraliser, à l'itération i , $T[i] = \min(T[i], \dots, T[n])$, et ainsi le tableau est trié de $T[1]$ à $T[i]$ si celui-ci est trié de $T[1]$ à $T[i-1]$. Pour $i = 1$, le tableau est trié puisqu'il n'y a qu'une seule valeur, donc par récurrence, T est trié quand $i = n$.

Exemple

3	7	2	6	5	1	4
1	7	2	6	5	3	4
1	2	7	6	5	3	4
1	2	3	6	5	7	4
1	2	3	4	5	7	6
1	2	3	4	5	7	6
1	2	3	4	5	6	7

Caractères

Ce tri n'est pas stable car il peut être amené à changer complètement l'ordre du tableau de base lors de l'échange du plus petit élément avec le premier. Cependant ce tri est en place puisque seul le tableau en entrée est modifié et utilisé pour obtenir le tableau trié en sortie.

Complexités

Temporelle La complexité de cet algorithme est la même dans tous les cas, en effet il effectue toujours $(n-1) + (n-2) + \dots + 2 + 1 = \frac{n(n-1)}{2}$ comparaisons. Soit une complexité en $\Theta(n^2)$.

Spatiale Ce tri étant stable, sa complexité spatiale est en $\Theta(n)$.

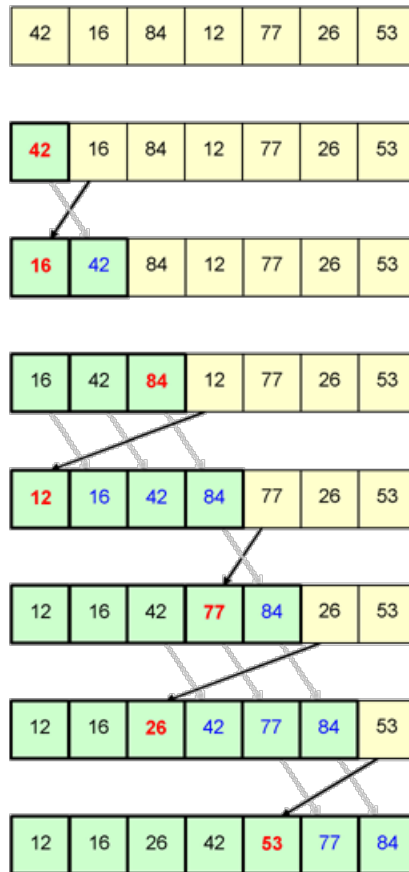
2.1.3 Tri par insertion (insertion sort)

Ce tri est assimilé à la manière dont on range les cartes d'un jeu que l'on nous distribue : nous prenons la carte et nous la rangeons à sa place dans le paquet.

Principe

Le principe est simple, au lieu de travailler sur le tableau entier, nous ne traitons qu'une partie de celui-ci. Nous prenons le dernier élément de cette partie et nous le réinsérons, en décalant les autres valeurs, à sa bonne place. A chaque tour de l'algorithme, nous traitons une partie de plus en plus grande de ce tableau jusqu'à ce que la partie du tableau soit le tableau entier.

Exemple



Caractères

Ce tri est stable et en place. En effet, il n'utilise qu'un seul tableau et conserve l'ordre d'apparition initial des éléments.

Complexités

Temporelle Dans le meilleur cas cet algorithme n'effectue qu'une comparaison par élément à insérer, on dispose de n éléments et $n - 1$ seront à insérer, il y a donc $n - 1$ comparaisons. Donc la complexité est en $\Omega(n)$. Dans le pire des cas, le cas où, dans chaque sous-partie que l'on va traiter, l'élément à insérer devra l'être à la première position, il faudra effectuer autant de comparaisons qu'il n'y a d'éléments dans chaque sous-partie du tableau. Le nombre de comparaisons correspond à la somme de la taille de toutes les sous-parties soit $1 + 2 + 3 + \dots + (n - 1)$ comparaisons, la somme de cette suite étant $\frac{n(n-1)}{2}$ ce qui donne donc une complexité en $O(n^2)$.

Dans le cas moyen il faut intégrer un calcul de probabilité, en considérant qu'à chaque itération la probabilité que l'élément en position i initialement sera à la position k une fois le tableau trié est de $\frac{1}{n}$ nous obtenons le nombre de comparaisons lors de la i -ème itération suivant : $\frac{i+1}{2} - \frac{1}{i}$. On note C_i ce nombre, la complexité moyenne du tri insertion correspond alors à la somme pour i allant de 2 à n de C_i . Ce qui donne $\frac{n^2+3n}{4} - H_n$ ou H_n est la série harmonique. La complexité est donc en $\Theta(n^2)$.

Spatiale La complexité spatiale est en $\Theta(1)$ car cet algorithme est en place, il n'utilise donc que très peu de mémoire externe.

2.1.4 Tri rapide (quick sort)

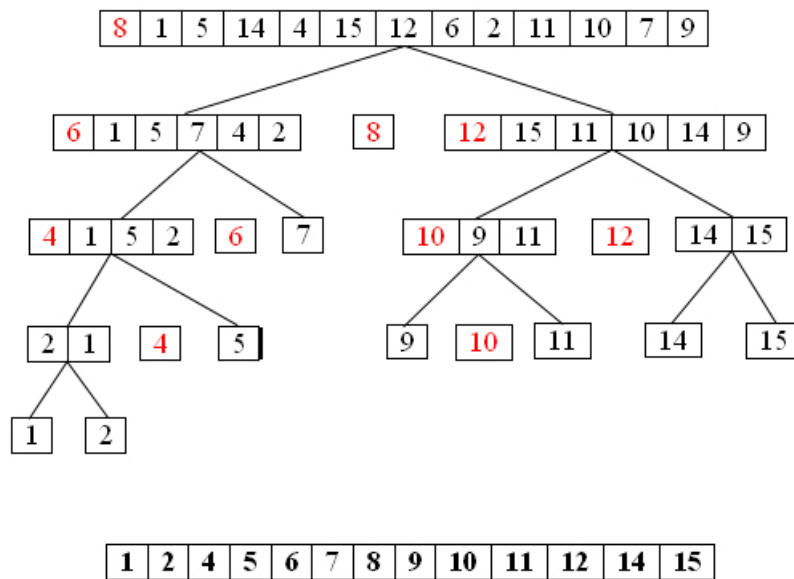
Le tri rapide (en anglais quicksort) est un algorithme de tri inventé par C.A.R. Hoare en 1961 et fondé sur la méthode de conception “diviser pour régner”.

Principe

Il convient de choisir un élément du tableau, nommé “pivot”. Ce choix est arbitraire. L’algorithme effectue tout d’abord un partitionnement, cela signifie qu’à la fin de cette opération, le tableau sera divisé en deux sous-tableaux, T_{inf} et T_{sup} tel que $\forall x \in T_{inf}$ et $\forall y \in T_{sup}$, $x \leq pivot \leq y$. Le pivot est mis à sa place finale dans le tableau (entre T_{inf} et T_{sup}) et l’algorithme est appliqué récursivement sur chaque sous-tableau, la récursion s’arrêtant lorsque que les sous-tableaux sont de taille 1.

Exemple

Voici un exemple d’exécution de l’algorithme, le pivot choisi est indiqué en rouge.



Caractères

Le tri rapide effectue un partitionnement du tableau pour pouvoir traiter les sous tableaux de celui-ci, c’est ce partitionnement qui définit les principales caractéristiques du tri rapide. En effet celui-ci préserve le caractère en place du tri rapide mais le rend non-stable car il empêche de garder nécessairement la position initiale des éléments.

Complexités

Pour la suite nous considérerons que le coût du partitionnement est de n , car chaque entier du tableau est comparé avec le pivot.

Temporelle - Meilleur cas Le meilleur cas est obtenu si la place finale du pivot est au milieu du tableau. Nous notons $T(n)$ le temps de calcul du tri rapide sur un tableau de n entiers. $T(n)$ correspond au coût du tri des deux moitiés du tableau et au coût de la partition des tableaux, nous obtenons alors

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n$$

Si l’on considère que n est une puissance de 2, nous pouvons poser $n = 2^p$. L’équation devient

$$T(2^p) = 2 \times T(2^{p-1}) + 2^p$$

C’est une récurrence, en calculant 2^{p-1} , nous obtenons

$$T(2^{p-1}) = 2 \times T(2^{p-2}) + 2^{p-1}$$

et nous remplaçons 2^{p-1} dans l'équation précédente, nous obtenons

$$T(2^p) = 2 \times (2 \times T(2^{p-2}) + 2^{p-1}) + 2^p = 2^2 \times T(2^{p-2}) + 2 \times 2^p$$

Ainsi par récurrence,

$$T(2^p) = 2^p \times T(2^{p-p}) + p \times 2^p = n \times T(1) + \log_2(n) \times n = n + n \times \log_2(n) = O(n \times \log_2(n))$$

Nous avons donc une complexité en $\Omega(n \times \log_2(n))$.

Maintenant, si n n'est pas une puissance de 2, il suffit de l'encadrer en deux puissance de 2, $2^p \leq n \leq 2^{p+1}$, la logique est la même que précédemment, donc nous obtenons une complexité en $\Omega(n \times \log_2(n))$ quelque soit la valeur de n .

Temporelle - Pire cas Le pire cas se produit, paradoxalement, lorsque le tableau en entrée est déjà trié. Dans ce cas à chaque partitionnement on obtient un tableau de taille $n-1$ et un autre de taille 1. On partitionne le tableau de taille $n-1$ et on obtient deux tableaux de tailles $n-2$ et 1 et ainsi de suite jusqu'à obtenir deux tableaux de taille 1. Comme tout à l'heure, $T(n)$ est égal à la somme des coûts des sous-tableaux ainsi que du coût de partitionnement, ce qui nous donne

$$\left. \begin{array}{l} T(n) = T(1) + T(n-1) + n = T(n-1) + n \\ T(n-1) = T(1) + T(n-2) + n-1 = T(n-2) + (n-1) \end{array} \right\} \Rightarrow T(n) = T(n-2) + (n-1) + n$$

Par récurrence,

$$T(n) = \sum_{i=1}^n i = \frac{n(n+1)}{2} = O(n^2)$$

On a donc une complexité en $O(n^2)$.

Temporelle - Cas moyen Pour le cas moyen on considère que la probabilité que chaque entier du tableau soit choisit comme pivot est égale à $\frac{1}{n}$. On obtient alors

$$T(n) = n + 1 + \frac{1}{n} \times \sum_{i=1}^n (T(i-1) + T(n-i))$$

En résolvant cette récurrence on obtient une complexité en $\Theta(n \times \log_2(n))$.

Spatiale La caractéristique en place de ce tri fait que sa complexité spatiale est en $\Theta(1)$.

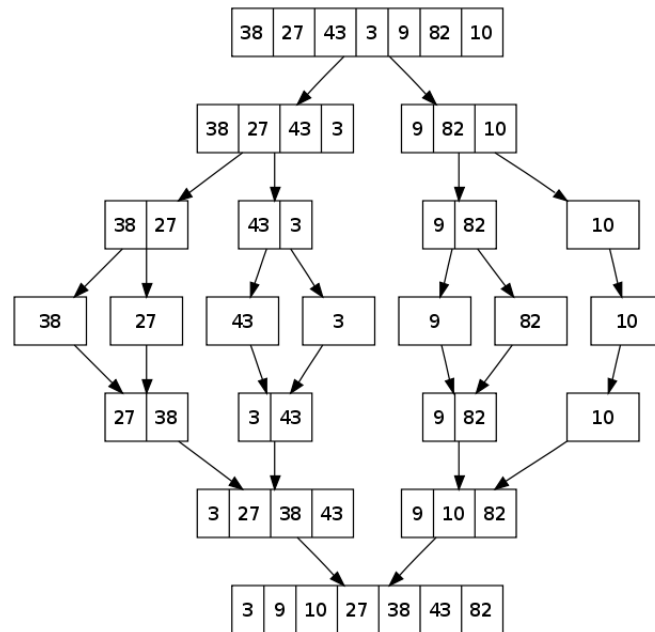
2.1.5 Tri fusion (merge sort)

Le tri fusion exploite directement le principe général du *divide and conquer*. C'est un tri très populaire qui, dans sa version la plus simple, est d'une efficacité comparable à celle du tri rapide.

Principe

L'algorithme divise le tableau en deux sous-tableaux de taille $\lfloor n/2 \rfloor$ et $\lceil n/2 \rceil$ et s'applique récursivement sur eux. La récursion s'arrête quand les sous-tableaux sont de taille 1. Ensuite, il les fusionne. Après cette opération, les valeurs du premier sous-tableaux sont toutes inférieures à celle du second, à l'instar du tri rapide. Cette étape nécessite un tableau auxiliaire pour être effectuée.

Exemple



Caractères

Le tri fusion est stable, par contre il n'opère pas en place car une zone de données temporaire de la même taille que les données en entrée est nécessaire au bon déroulement de l'algorithme.

Complexités

Temporelle La complexité temporelle du tri fusion est comparable à celle du tri rapide, en effet l'algorithme divise le tableau en 2 sous-tableaux de même taille à chaque récursion quelque soit le tableau, nous nous retrouvons avec une complexité égale à celle du tri rapide dans le meilleur cas,

$$T(n) = 2 \times T\left(\frac{n}{2}\right) + n$$

L'ajout de n correspond ici au temps nécessaire à la fusion des deux sous-tableaux.

Spatiale L'espace supplémentaire nécessaire aux tableaux auxiliaires entraîne une complexité spatiale en $\Theta(n)$.

2.1.6 Tri par tas (heap sort)

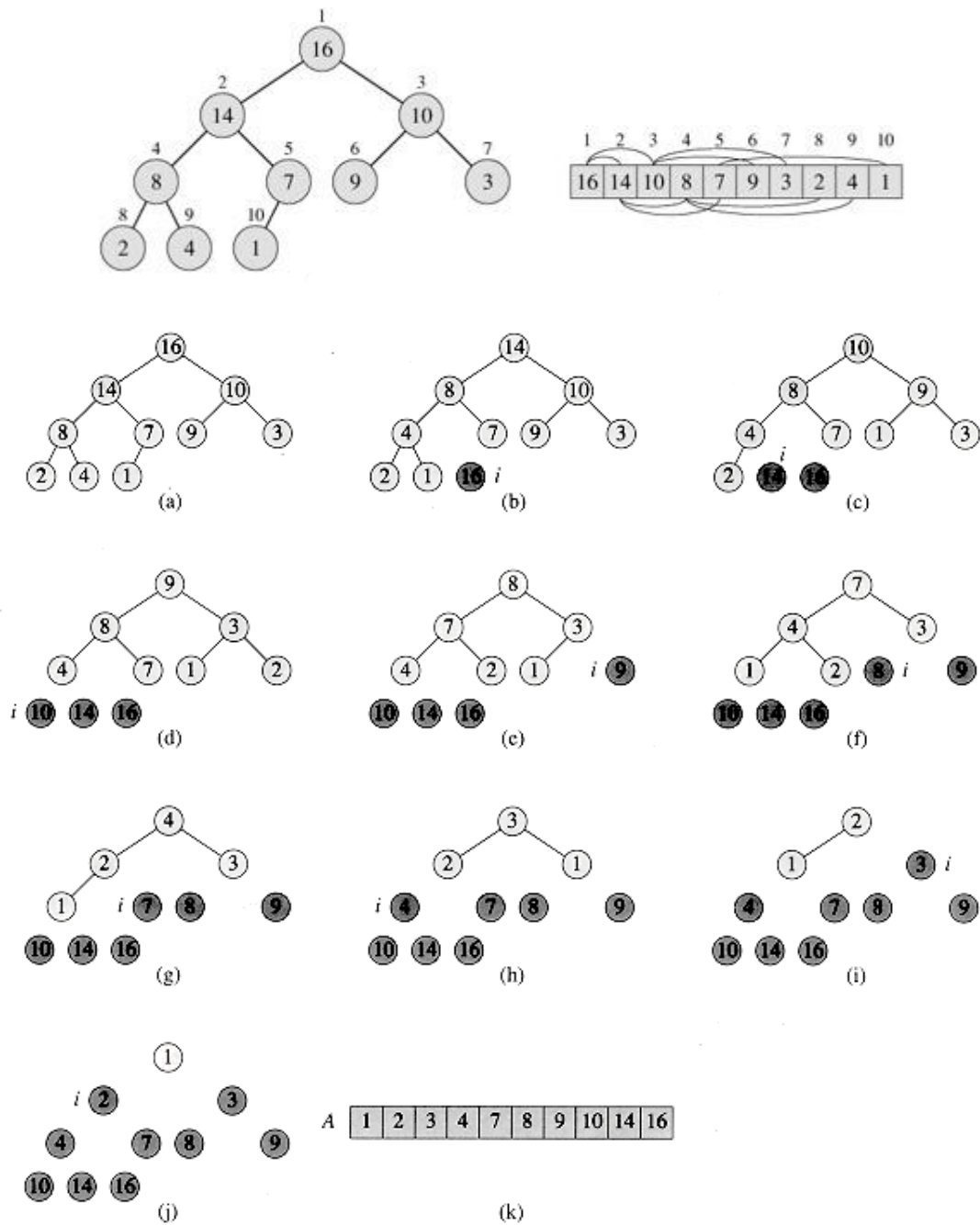
Un tas est une structure de données qui peut être représentée par un arbre complet à gauche et qui vérifie la condition d'ordre d'un tas : la clé d'un noeud est supérieure ou égale à la clé de chacun de ses fils. Le tri par tas utilise cette notion et a été inventé en 1964 par J.W.J Williams.

Principe

Le fonctionnement du tri par tas peut être divisé en deux phases.

Dans la première phase, l'algorithme transforme les données en entrée en un tas, celui-ci étant souvent stocké dans un tableau. Le premier élément est la racine, le deuxième et le troisième sont les deux descendants du premier élément, etc. Dans la seconde phase, un tableau trié est créé en enlevant le plus gros élément du tas répétitivement et en l'insérant dans le tableau. Le tas est mis à jour après chaque extraction de sorte à maintenir sa structure de tas. Une fois tous les éléments extraits du tas le résultat est un tableau trié.

Exemple



Caractères

Ce tri ne nécessite pas d'allocation de zone mémoire supplémentaire, il est donc en place. Cependant sa stabilité est compromise par le transfert entre tas et tableau.

Complexités

Temporelle La complexité du tri par tas est clairement $\Theta(n \times \log_2(n))$. Il y a deux boucles séparées. Chacune consiste en une boucle de durée $\Theta(\log(n))$ imbriquée dans une boucle de durée $\Theta(n)$.

Spatiale Sa complexité spatiale est en $\Theta(1)$, car ce tri est en place.

2.2 Tris sans comparaisons

2.2.1 Tri par base (radix sort)

Ce tri est fort intéressant d'un point de vue historique, il était celui utilisé pour trier les cartes perforées dans les débuts de l'informatique. Il a été inventé en 1887 par Herman Hollerith sur tabulatrice, une machine répondant donc à des programmes sur ces fameuses cartes.

Principe

Imaginons comme à chaque fois que l'on cherche à trier n entiers de longueur maximale k . Le tri par base range les éléments en fonction des chiffres de chaque valeur, par exemple le chiffre des unités. Tous les entiers ayant même unité seront rangés au même endroit, dans une pile. De cette description, nous voyons tout de suite qu'il nous faut 10 piles, afin de pouvoir mettre chaque entier à un endroit (les chiffres allant de 0 à 9). Une fois tous les entiers rangés en piles, il ne reste plus qu'à les sortir les valeurs dans le même ordre que lors de leur insertion en partant la pile 0 et en allant vers la pile 9 (une structure de file est bien évidemment plus appropriée informatiquement parlant, ici le terme pile est plus général). Il ne reste plus qu'à faire le même procédé pour le chiffre des dizaines, des centaines et ainsi de suite en itérant jusqu'à k .

Il reste cependant le cas des nombres négatifs à gérer. En effet, si l'on applique l'algorithme avec ces derniers, ils seront tout simplement considérés comme positifs. La solution la plus simple est d'enlever d'abord tous les nombres négatifs des piles, puis les positifs. Seulement, si nous commençons à la pile 0 pour les négatifs, vers la 9, il y aura un problème. L'ordre des valeurs en négatif est opposé à celui en positif, de ce fait, il est nécessaire de commencer à la pile 9, en allant vers la pile 0, en extrayant tous les nombres négatifs, puis de repartir de la pile 0 vers la pile 9 en extrayant tous les nombres positifs.

Exemple

4925	2623	2623	2345	2345
2623	4524	4524	2346	2346
2346	4925	4925	2436	2436
8675	8675	2436	4524	2623
3657	4545	4545	4545	3657
4546	2345	2345	4546	4524
7566	2346	2346	7566	4545
4545	4546	4546	2623	4546
4524	7566	3657	3657	4925
2345	2436	7566	8675	7566
2436	3657	8675	4925	8675

Caractères

Ce tri n'est donc pas en place, il utilise beaucoup de mémoire externe pour stocker les valeurs en pile, mais il est stable, car les valeurs sont sorties des piles dans l'ordre dans lequel elles sont entrées.

Complexités

Temporelle La complexité temporelle de ce tri est tout le temps la même, cela paraît assez évident au vu du principe, les valeurs sont parcourues k fois et il y en a n , la complexité temporelle est donc $\Theta(k \times n)$.

Spatiale En ce qui concerne la complexité spatiale, nous voyons que ce tri nécessite plusieurs piles de taille n , pour prévoir tous les cas, ce qui nous fait $n \times m$ emplacements mémoire, si m est le nombre de piles nécessaires. Nous en déduisons que la complexité spatiale est en $\Theta(m \times n)$.

En complément

Nous pouvons également faire l'exécution dans l'ordre inverse, l'algorithme existe dans les deux versions, la première est **Least Significant Digit Radix Sort**, la seconde **Most Significant Digit Radix Sort** mais elle a une complexité un peu différente que nous n'avons pas étudié ici. Pour plus de détails, se référer à [1] et [2].

2.2.2 Tri comptage (counting sort)

Le tri comptage s'apparente à un recensement des valeurs à trier. Il a été inventé par Harold H. Seward en 1954.

Principe

Cet algorithme compte le nombre d'apparition de chaque valeur des tableaux à trier. Cela pose donc une contrainte importante, la connaissance au préalable des valeurs à traiter, ou plutôt l'intervalle dans lequel elles se trouvent. Ensuite, l'algorithme va prendre connaissance de chaque valeurs du tableau et la recenser. Il se sert donc d'un tableau auxiliaire pour comptabiliser les nombres d'apparitions, de taille k , celle de l'intervalle. Ensuite, il le parcourt de l'indice 1 à k , et pour chaque $i \in \{1, \dots, k\}$, il fait apparaître la valeur i dans le tableau final le nombre de fois qu'elle a été comptabilisée.

Exemple

Tableau en entrée

0	4	2	2	0	0	1	1	0	1	0	2	4	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---

Tableau de comptage

0	1	2	3	4
5	3	4	0	2

Tableau en sortie

0	0	0	0	0	1	1	1	2	2	2	2	4	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---

Caractères

Cet algorithme n'est pas en place, il utilise une structure externe pour sauvegarder les apparitions de chaque valeur, la mémoire nécessaire est donc conséquente. Dire qu'il est stable n'a pas vraiment de sens, car l'agencement des valeurs au sein du tableau résulte du parcours du tableau de comptage, et non celui du tableau de départ.

Complexités

Temporelle Si l'on trie n valeurs comprises dans un intervalle de taille k , la complexité temporelle est en $\Theta(n+k)$, car l'algorithme parcourt le tableau des valeurs de départ, puis celui de comptage. Il est donc impossible de faire mieux ou moins bien.

Spatiale La complexité spatiale est en $\Theta(k)$, le tableau de comptage étant de taille k , nécessaire à l'algorithme dans tous les cas.

2.2.3 Tri par paquets (bucket sort)

Cet algorithme utilise la répartition des nombres dans un intervalle de façon uniforme. Mieux les valeurs seront réparties dans ce dernier, plus l'algorithme sera performant.

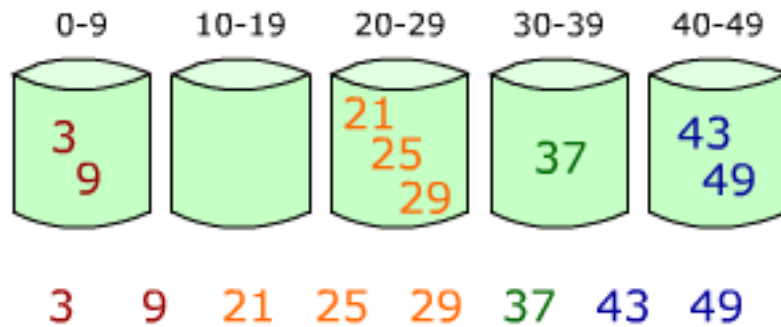
Principe

Tout d'abord, l'algorithme divise l'intervalle des valeurs à trier en n sous-intervalles de même longueur $I_0, \dots, I_{n-1}$. Si les valeurs sont comprises dans l'intervalle $[a, b[$, chaque sous-intervalle $I_k, \forall k \in \{0, \dots, n-1\}$ est déterminé grâce au calcul

$$I_k = [a + \frac{k(b-a)}{n}, a + \frac{(k+1)(b-a)}{n}[.$$

Puis il répartit ces dernières dans ces sous-intervalles, que l'on appelle paquets. Chaque paquet est trié séparément avec un algorithme de tri auxiliaire. Il suffit ensuite de concaténer ces paquets une fois qu'ils ont été triés.

Exemple



Caractères

Les caractères en place et stable de cet algorithme sont impossibles à établir réellement. En effet, ceux-ci dépendent de l'algorithme de tri auxiliaire pour trier les paquets. Si ce dernier présente ces deux caractères, alors le tri par paquets les présentera de même, et inversement.

Complexités

Ici aussi, il est difficile d'affirmer une complexité figée, car elle dépend de cet algorithme auxiliaire. Cependant nous pouvons distinguer le meilleur et le pire cas.

Temporelle - Meilleur cas Ce cas se présente si chaque valeur de l'intervalle correspond à un seul paquet. Un paquet de une valeur est trié en $O(1)$, quelque soit l'algorithme auxiliaire. De ce fait, la complexité de l'algorithme est en $\Omega(n)$.

Temporelle - Pire cas Nous avons ce cas si toute les valeurs à trier se retrouvent dans un seul paquet. La complexité sera donc celle de l'algorithme auxiliaire, $O(n^2)$ si l'on utilise le tri insertion, $O(n \times \log_2(n))$ avec le tri fusion.

Spatiale Comme précédemment, cette complexité dépend de l'algorithme auxiliaire utilisée.

A propos de l'algorithme auxiliaire

Il est intéressant de se pencher un tout petit peu plus sur l'algorithme auxiliaire utilisé. En effet, pourquoi ne pas utiliser directement le tri par paquets comme tel ? Il n'y aurait ainsi aucune dépendance avec un autre algorithme. Cependant, ceci n'est pas envisageable. Reprenons la complexité dans le pire cas... nous avons vu que celle-ci est obtenue lorsque toutes les valeurs à trier se retrouvent dans le même paquet. De ce fait, si nous réutilisons de manière récursive l'algorithme sur ce paquet, il produira le même résultat. Donc nous ne pouvons pas envisager de nous servir de l'algorithme principal comme algorithme auxiliaire.

2.3 Réseaux de tris (sorting networks)

2.3.1 Généralités

Intéressons-nous maintenant à une dernière catégorie de tris, les réseaux de tris. Ceux-ci diffèrent des deux classes de tris précédentes par le fait qu'ils peuvent être utilisés sur des ordinateurs effectuant des opérations en parallèle. Tous les algorithmes vus jusqu'ici n'autorisaient qu'une opération à la fois, ceux-ci permettent donc d'en faire plusieurs en même temps. Un autre point important, les réseaux de tris sont basés exclusivement sur des comparaisons de valeurs, il est donc impossible de créer des algorithmes tels que le tri comptage ou par base.

Toute cette partie a été largement inspirée de [2], où l'on peut trouver les démonstrations des propriétés que nous verrons. Se référer donc à cet ouvrage pour une connaissance plus approfondie du sujet.

Voyons maintenant la brique de base de tout réseau de tri. Comme ceux-ci sont basés sur des comparaisons, il est évident que nos réseaux seront des réseaux de comparaisons. Nous aurons donc besoin de **comparateurs**, permettant de comparer (comme son nom l'indique) deux valeurs puis de les échanger en fonction de l'ordre des valeurs souhaité à la fin du tri. Un comparateur croissant effectue l'opération comme présentée dans la figure suivante.

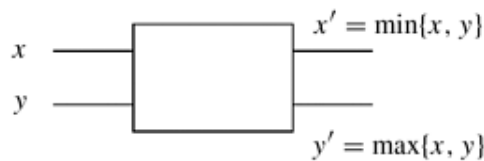


FIGURE 2.1 – Comparateur croissant

Pour des raisons de pratique nous utiliserons une représentation moins encombrante, ainsi le comparateur sera représenté comme suit.

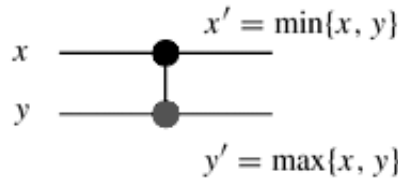


FIGURE 2.2 – Comparateur croissant représenté en trait vertical

Un réseau de comparaisons est représenté avec des **câbles**, qui transmettent des valeurs d'un endroit à un autre. Ceux-ci peuvent relier des sorties de comparateurs aux entrées d'autres comparateurs, sinon ils sont appelés **câbles d'entrée** et **câbles de sortie** du réseau. Un réseau de comparaisons contiendra n câbles d'entrée $x_1, x_2, x_3, \dots, x_n$ qui fourniront les valeurs à comparer tout au long du réseau et aura n câbles de sortie $y_1, y_2, y_3, \dots, y_n$ qui nous donneront le résultat du réseau. Nous pouvons parler de **séquences d'entrée** $\langle x_1, x_2, x_3, \dots, x_n \rangle$ et de **séquences de sortie** $\langle y_1, y_2, y_3, \dots, y_n \rangle$. Nous considérerons aussi que chaque comparaison se fait en temps constant, donc en $O(1)$.

Il est important aussi de parler de la **profondeur** d'un câble. Un câble d'entrée a une profondeur de 0. Ensuite, à chaque comparateur, il faut appliquer le calcul suivant :

$$d = \max(d_x, d_y) + 1$$

d : profondeur du câble au niveau du comparateur considéré

d_x : profondeur du câble du côté de l'entrée x

d_y : profondeur du câble du côté de l'entrée y

La profondeur d'un câble d'entrée est de 1. Ainsi cette valeur permet de savoir le nombre maximum de comparateurs que traversera une valeur entre un câble d'entrée et un câble de sortie, et donne une idée de l'efficacité du réseau.

Un **réseau de tri** est simplement un réseau de comparaisons dont la séquence de sortie est monotone, donc telle que $y_1 \leq y_2 \leq y_3 \leq \dots \leq y_n$ pour toute séquence d'entrée.

Il nous reste juste à voir un dernier point concernant ces réseaux, le **principe zéro-un**. Ce dernier nous dit que si un réseau de tri fonctionne correctement sur les valeurs $\{0, 1\}$ alors il fonctionnera sur n'importe quelles autres valeurs, entières ou non. Ce principe est fort appréciable car il permet de se concentrer uniquement sur les séquences de 0 et de 1 lorsque l'on cherche à créer un réseau de tri. S'il fonctionne avec ces deux valeurs, nous sommes assurés qu'il fonctionnera sur d'autres. Un gain de temps considérable, en somme. Cette propriété a été découverte par W. G. Bouricius, en 1954.

Pour terminer cette section, voici un exemple de réseau de tri très simple avec la profondeur des câbles indiquée à chaque comparateur :

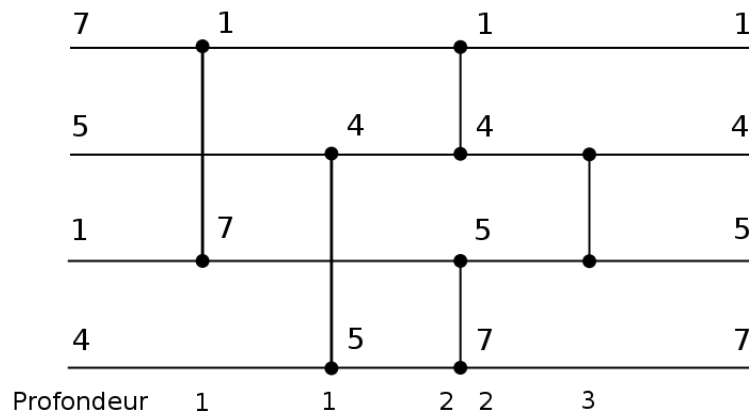


FIGURE 2.3 – Réseau de tri de 4 valeurs

Il est intéressant de chercher à refaire des algorithmes de tri classique à la mode des réseaux de tri. Par exemple, voici les tri à bulles et tri insertion :

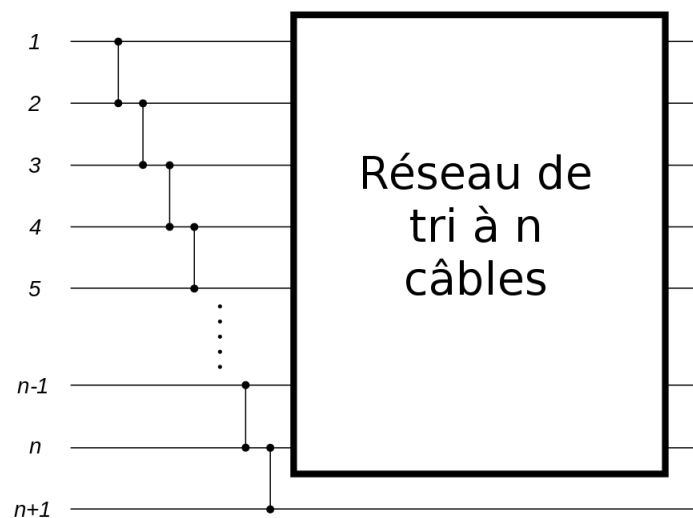


FIGURE 2.4 – Tri à bulles

Nous voyons bien ici le principe de repousser la plus grande valeur du réseau vers le câble le plus bas.

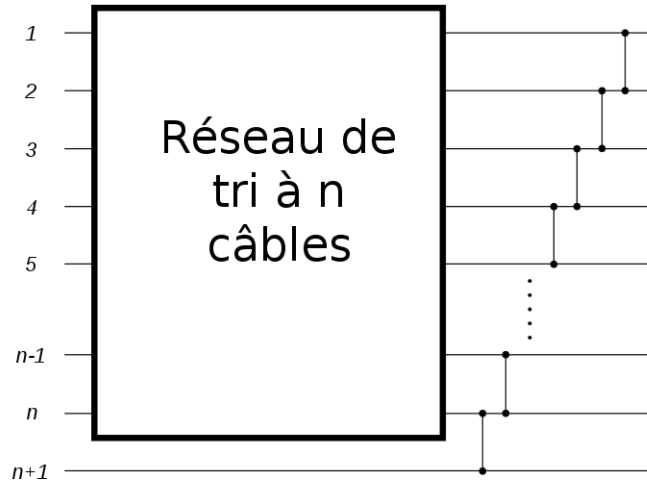


FIGURE 2.5 – Tri insertion

Pareil ici, nous comprenons aisément que nous insérons la valeur considérée à chaque itération au bon endroit.

Petit remarque intéressante, ces deux réseaux que l'on présente différemment sur ces schémas peuvent se représenter exactement de la même manière, voici un exemple :

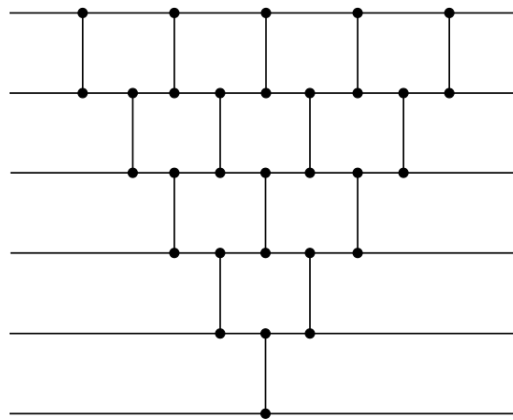


FIGURE 2.6 – Tri insertion et tri à bulle de 6 entrées

2.3.2 Réseau de tri bitonique (bitonic sorter)

L'objectif de ce réseau est de trier des **séquences bitoniques**, qui sont des séquences de nombres croissant de façon monotone puis décroissant de la même manière. Si en effectuant une rotation sur une séquence, celle-ci répond au critère énoncé, alors nous la considérons également bitonique. Par exemple, $\langle 10, 5, 4, 6, 8 \rangle$ est une séquence bitonique. Comme nous allons uniquement travailler sur des séquences composées de 0 et de 1, celles-ci seront toutes de la forme $0^a 1^b 0^c$ ou $1^a 0^b 1^c$ avec $a, b, c \geq 0$. Une séquence composée uniquement de 0 ou de 1 est dite **pure**.

Principe

Afin de créer ce réseau, nous aurons besoin d'un module appelé séparateur. Ce dernier est composé de $n/2$ comparateurs, qui prennent en entrées un câble i et le câble $i + n/2$ pour $i = 1, \dots, n/2$. Ainsi, si sa séquence d'entrée est bitonique alors sa séquence de sortie aura les propriétés suivantes : sa moitié haute ainsi que sa moitié basse seront bitoniques et les éléments de la moitié haute seront tous plus petits que ceux de la moitié basse. Nous appellerons ce module $SEPARATEUR[n]$.

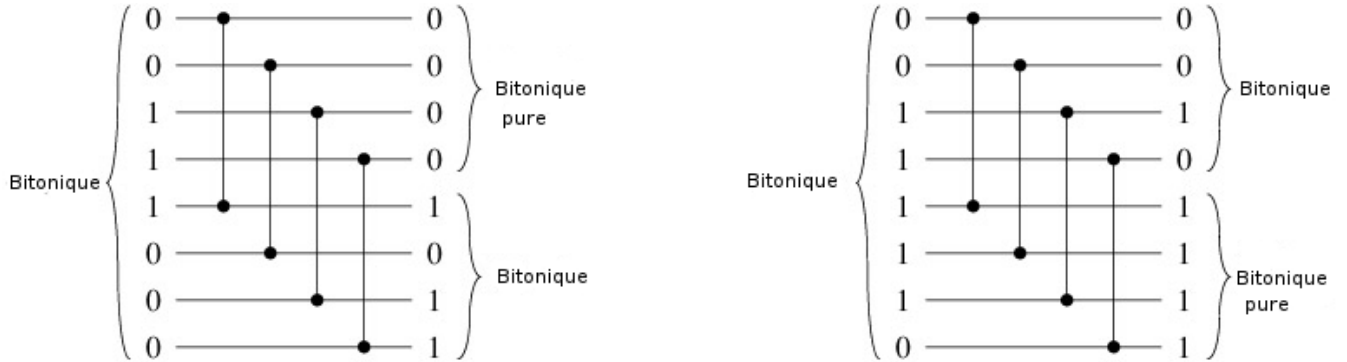


FIGURE 2.7 – Exemples avec $SEPARATEUR[8]$

D'après la définition du module de séparation, nous pouvons maintenant combiner les séparateurs de façon récursive, afin de trier toute la séquence. En effet, les éléments de la moitié haute étant tous plus petits que ceux de la moitié basse, nous aurons trié les valeurs à la fin de la récursion. Créons donc $TRIEUSE_BITONIQUE[n]$ qui sera composée d'un $SEPARATEUR[n]$ puis de deux $TRIEUSE_BITONIQUE[n/2]$, tel que présenté sur la figure suivante.

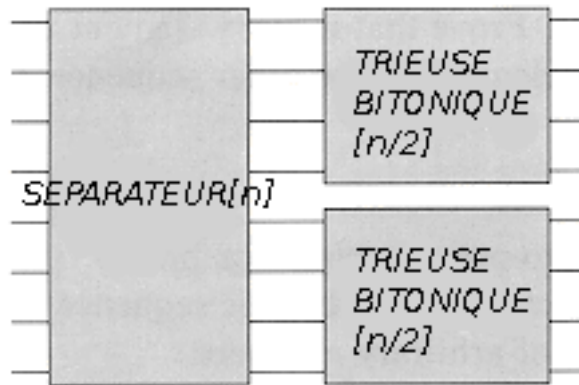
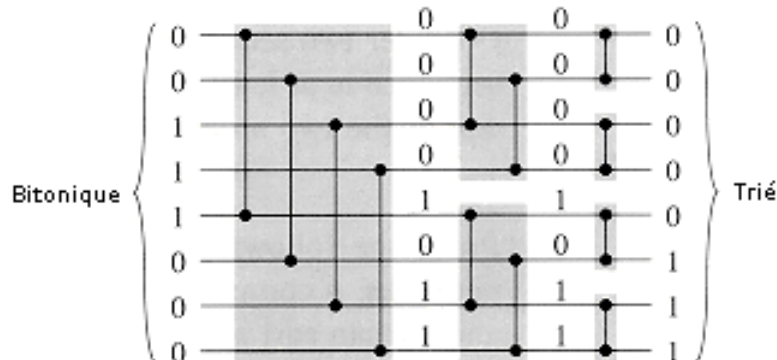


FIGURE 2.8 – $TRIEUSE_BITONIQUE[n]$

Exemple



Caractères

Ce tri est en place, nous sommes en présence d'un algorithme basé seulement sur des comparaisons-échanges. Mais il n'est pas stable, car deux valeurs peuvent perdre leur position relative si elles sont comparées avec des valeurs plus petites ou plus grandes qu'elles.

Complexité

Temporelle La profondeur du réseau permet de connaître la complexité. En effet, cela représente le nombre maximal de comparateur que traverse une entrée pour atteindre la sortie. Cette profondeur nous est donnée par l'équation :

$$D(n) = \begin{cases} 0 & \text{si } n = 1 \\ D(n/2) + 1 & \text{si } n = 2^k, k \geq 1 \end{cases}$$

dont la solution est $D(n) = \log_2(n)$. La complexité temporelle est donc en $\Theta(\log_2(n))$. Il n'y a pas de meilleur cas, ni de moins bon car dans tous les cas, il faudra attendre au moins, et au plus, $\log_2(n)$ comparaisons un câble ou plus. Le résultat sera donc livré après ces $\log_2(n)$ opérations.

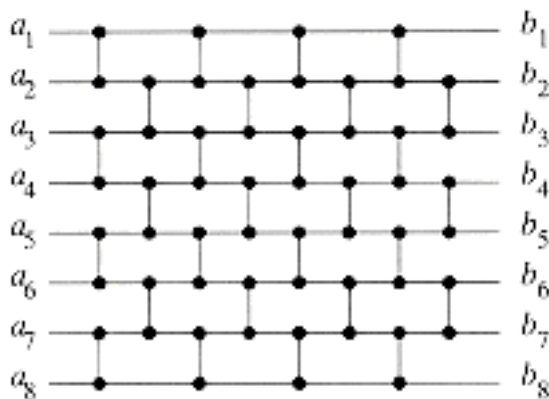
Spatiale En ce qui concerne la complexité spatiale, au vu du fait qu'il y a très peu de mémoire utilisée, celle-ci est en $\Theta(1)$

2.3.3 Réseau de tri pair - impair (odd-even merge sort)

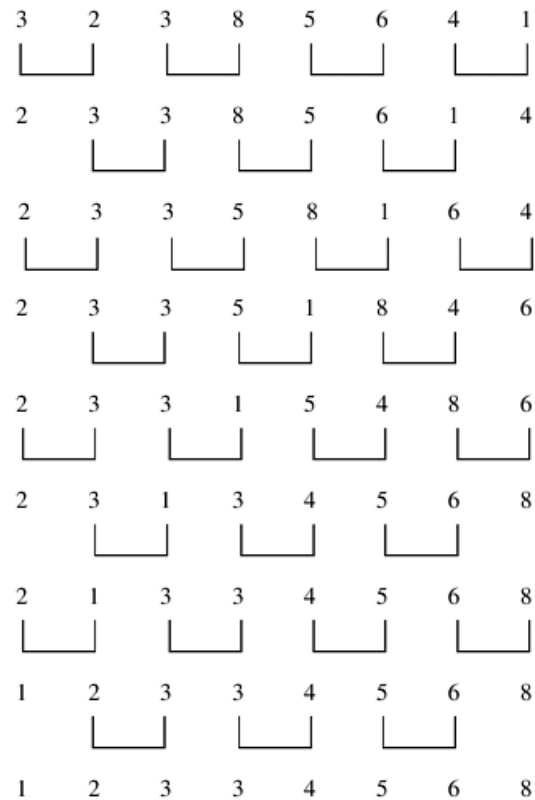
Cet algorithme se pose comme une variante du tri à bulles.

Principe

Nous avons vu que ce dernier compare 2 à 2 les valeurs à trier dans le tableau, or cela pose un problème pour paralléliser les comparaisons-échanges. En effet, il est nécessaire d'attendre qu'une comparaison aux indices i et $i+1$ soient faites pour pouvoir faire celle aux indices $i+1$ et $i+2$. Nous avons donc une dépendance (de données). L'idée du tri-impair est donc de comparer les valeurs 2 à 2 en enlevant ces dépendances. Pour cela, il suffit de comparer les valeurs d'indices pairs avec leur suivant dans un premier temps, puis de faire la même chose avec les indices impairs et leur suivant. De la même manière que pour le tri à bulles, il faut faire ces opérations $n - 1$ en tout. C'est-à-dire que l'on fera $(n - 1)/2$ fois les comparaisons d'indices pairs avec leur suivants, et la même chose pour les indices impairs. Ce tri peut être représenté comme un réseau de cette manière :



Exemple



Caractères

Comme le tri à bulles, celui-ci est en place car il n'utilise que très peu de mémoire externe pour échanger les valeurs. Il est également stable, car deux valeurs ne seront pas échangées si elles sont égales, elles auront donc la même position relative que celle de départ.

Complexité

Temporelle Nous nous considérerons sur une machine massivement parallèle pour calculer cette complexité. Si toutes les comparaisons se font en parallèle alors un parcours de tableau est réalisé en $\Theta(1)$. Puisque nous faisons $n - 1$ parcours, nous nous retrouvons avec une complexité en $\Theta(n)$.

Spatiale Celle-ci est en $\Theta(1)$ car le tri est en place.

Chapitre 3

Résultats expérimentaux

Après toutes ces explications, il est temps de passer à la partie pratique, celle dans laquelle donc nous avons implémenté (ou implanté) les algorithmes de tris présenté précédemment. Ainsi nous présenterons les résultats en temps de chaque algorithme sous forme de graphiques. Les algorithmes et codes sources utilisés sont présentés dans les chapitres suivants. Pour situer les choses, il semble important de donner les conditions de tests.

Caractéristiques de la machine Nous avons utilisé une machine composée de :

- un processeur quatre coeurs **AMD A6-3410MX APU with Radeon(tm) HD Graphics**, cadencés à 1.6 GHz,
- 6 Go de mémoire vive,
- un système d’exploitation Linux, plus précisément Ubuntu 12.04 LTS 32 bits.

Caractéristiques des données Les valeurs sont générées de manière aléatoire dans un intervalle. Et de là découle un problème, les temps nécessaires à chaque algorithme diffère d’une exécution sur l’autre à cause de la répartition des données. Pour pallier à cela, nous avons fait **une moyenne de 8 exécutions par tris**. Bien que cela ne suffise pas vraiment pour obtenir des temps d’exécutions stables (qui ne diffèrent presque pas d’une exécution sur l’autre), nous n’avons pas pu nous autoriser plus à cause des limitations matérielles de la machine de tests. Cependant, ces temps restent tout de même assez stables, nous n’obtenons pas énormément d’écarts entre différentes exécutions.

Outils de mesures Nous nous sommes servis de GCC, fourni sur n’importe qu’elle distribution Ubuntu, pour la compilation des codes sources. Pour la mesure des temps d’exécution, nous avons utilisé les **struct timeval** qui sont dans la bibliothèque standard du C. Etant donné que cette possibilité est offerte avec le langage, nous pensons que ces mesures sont fiables. De plus, ces dernières sont précises jusqu’à la microseconde.

3.1 Tri par comparaisons

Pour le graphique qui suit, nous avons utilisé les algorithmes sur des tableaux de taille 0 à 30000. Pour des raisons matérielles, nous ne pouvons aller guère plus loin, car les algorithmes basiques (de complexités temporelles en $\Theta(n^2)$) sont trop longs à être exécutés à partir de 30000. L’intervalle des valeurs influant très peu la vitesse des algorithmes par comparaisons, celui-ci est $[-1024, 1024]$ pour tous ces tris.

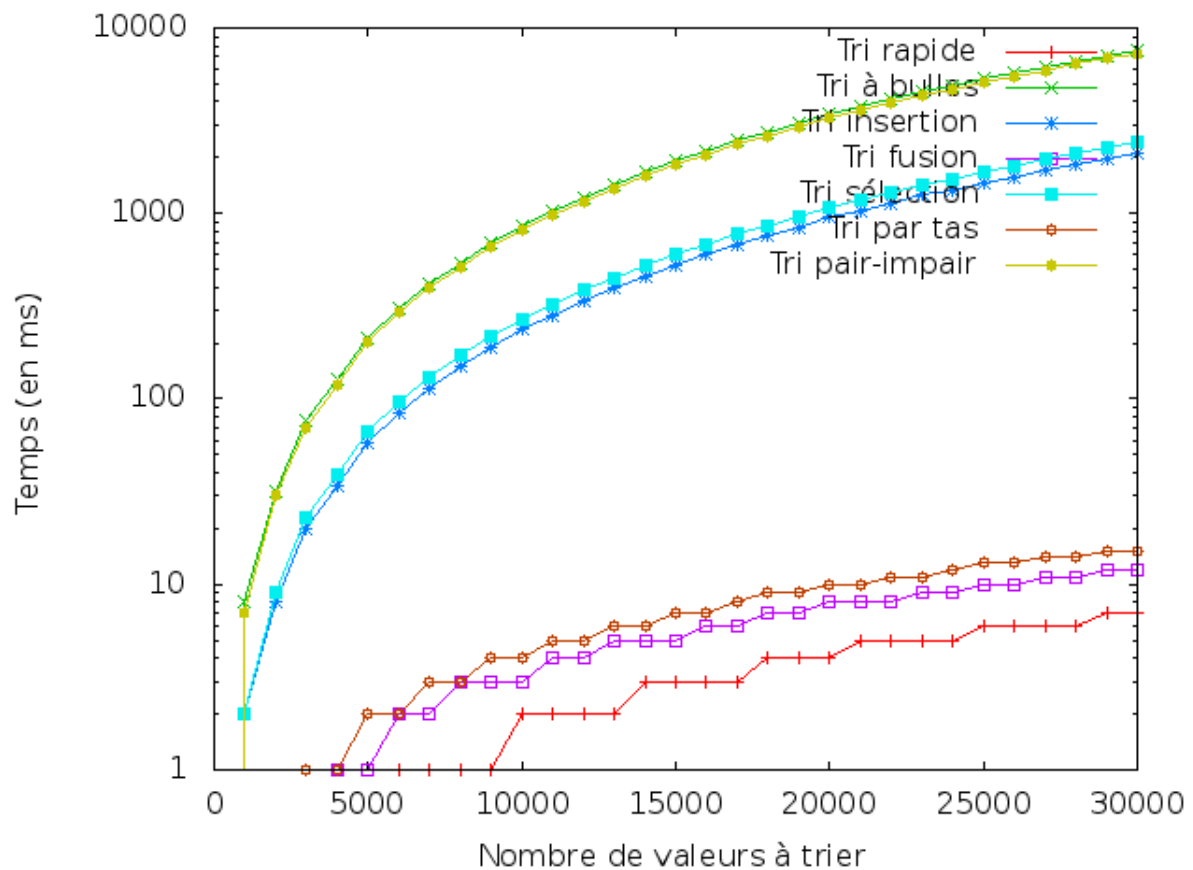


FIGURE 3.1 – Comparatifs de temps de tous les tris par comparaisons

3.2 Tri par comparaisons rapides

Les graphiques ci-dessus présentent tous les tris par comparaisons que nous avons vu. L'inconvénient est que l'on ne voit pas très bien, entre les tris plus rapides, quels sont les plus performants. Voici donc un graphique qui compare les tris ayant une complexité temporelle en $\Theta(n \times \log_2(n))$. Nous avons fait varier le nombre de valeurs à trier de 0 à 1000000.

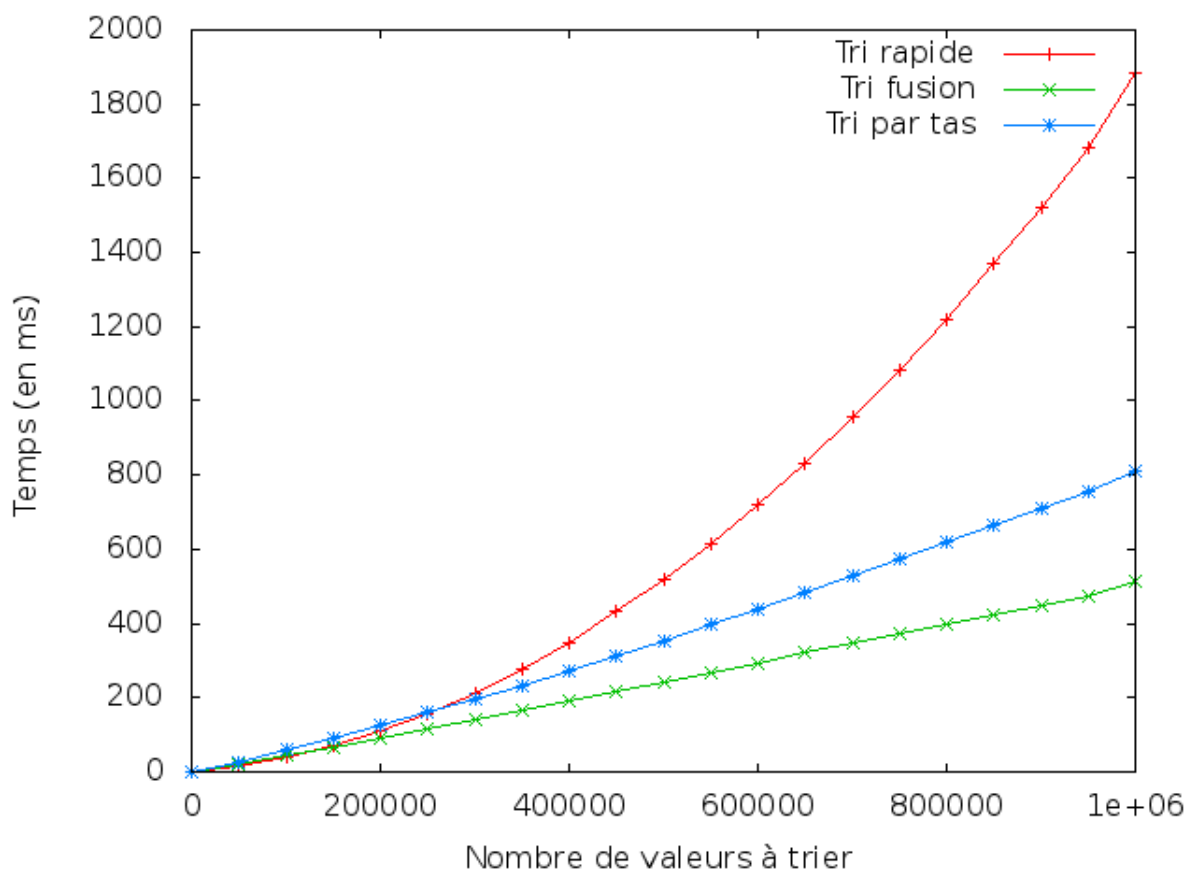


FIGURE 3.2 – Comparatifs de temps des tris rapides par comparaisons

3.3 Tri sans comparaisons

Pour finir, regardons les tris sans comparaisons. Nous étudions des tris qui se basent sur l'intervalle des valeurs, aussi allons-nous montrer deux graphiques avec des intervalles différents. Un très petit et un autre au limite de ce que peut supporter l'ordinateur de test en terme d'allocation mémoire. En effet, le tri comptage nécessitant un tableau proportionnel à l'intervalle des valeurs à recenser, l'allocation mémoire a échoué quand ce dernier était trop grand. Puisque les algorithmes sont très influencés par les intervalles de valeurs, et que celles-ci sont générés aléatoirement, il nous a paru important de doubler le nombre d'exécution par tri afin d'avoir des temps plus stables.

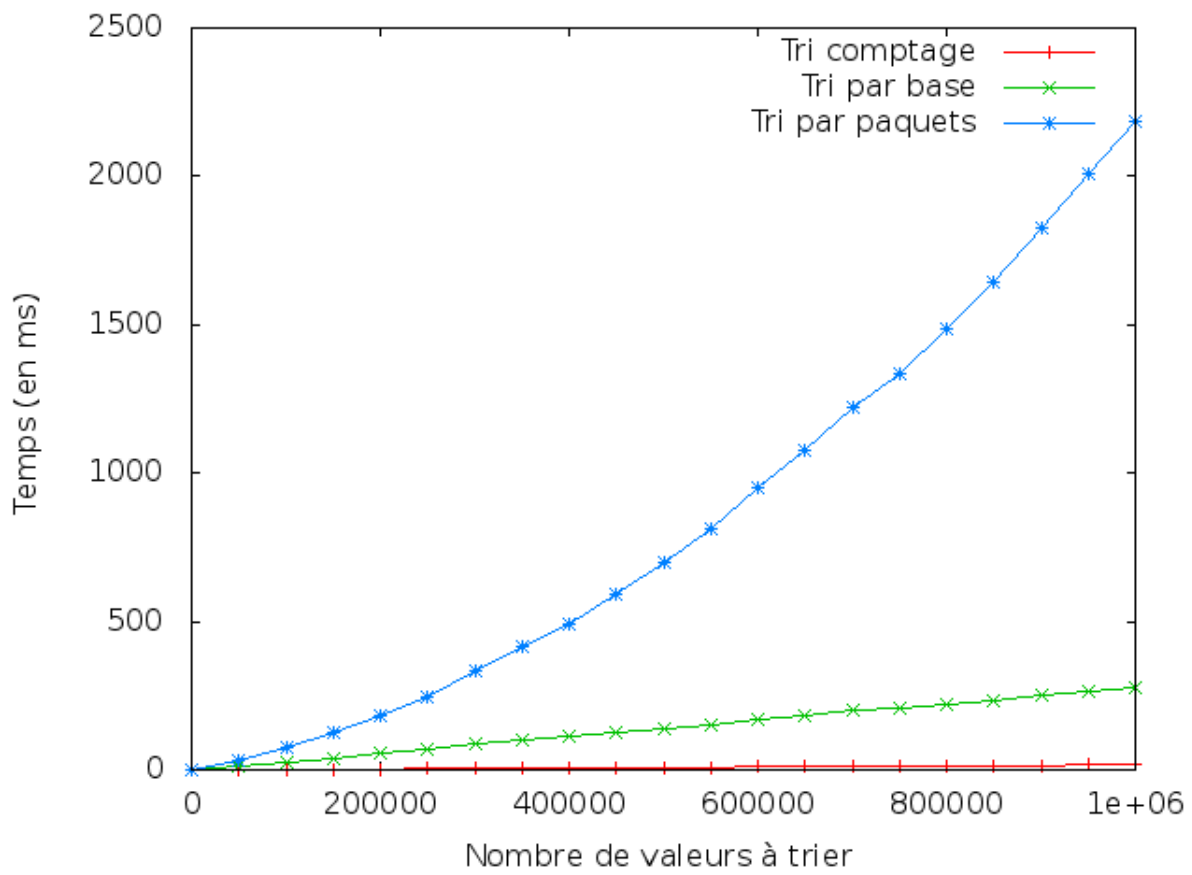


FIGURE 3.3 – Comparatifs de temps des tris sans comparaisons sur l'intervalle $[-1024, 1024]$

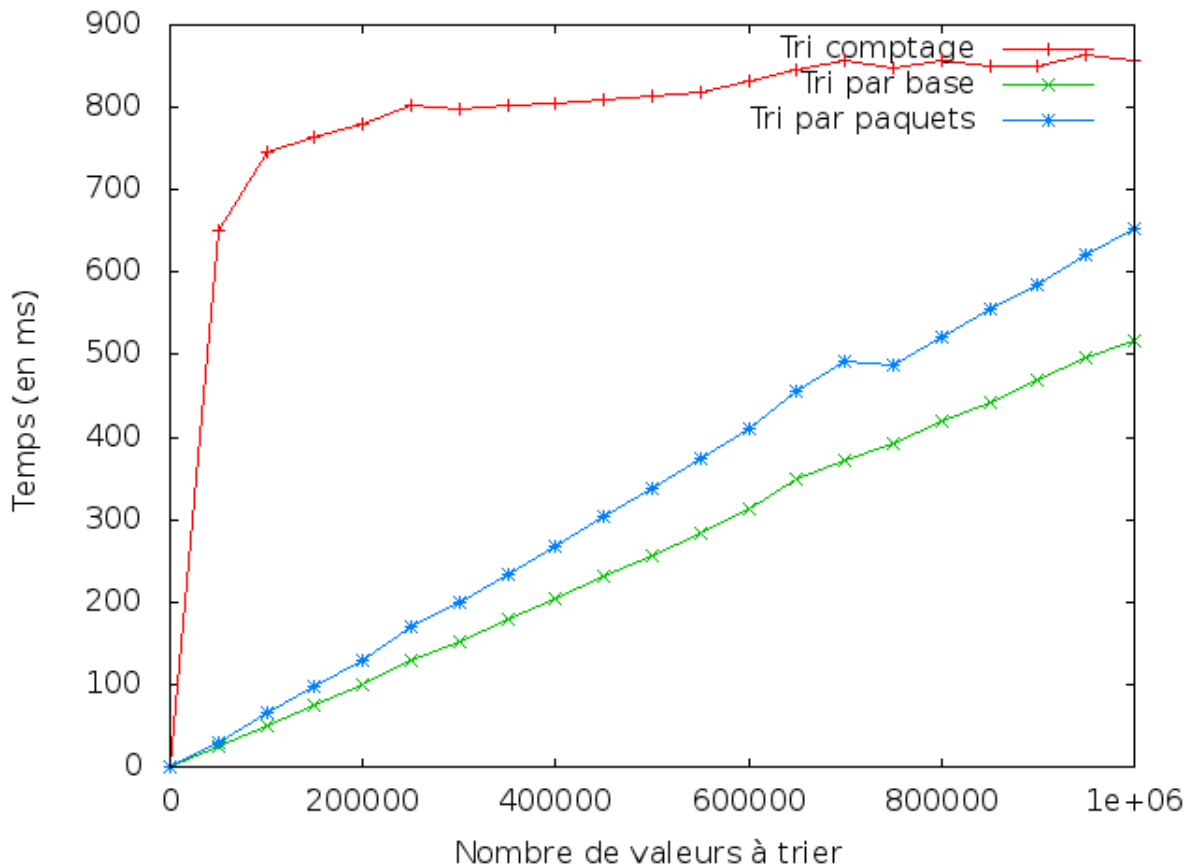


FIGURE 3.4 – Comparatifs de temps des tris sans comparaisons sur l'intervalle $[-2^{25}, 2^{25}]$

3.4 Interprétation

Le premier graphique nous informe que la complexité temporelle d'un algorithme est intéressante à prendre en compte, mais que dans les faits, certains algorithmes sont plus intéressants que d'autres. Le tri à bulles et sa variante, par exemple, apparaissent comme les moins performants, tandis que le tri sélection et le tri insertion sont plus intéressants. Néanmoins, leur vitesse respective croît de la même manière en fonction du nombre de valeurs à traiter. Ils restent peu pratiques pour les grands tableaux de valeurs.

Nous constatons que la variante parallèle du tri à bulles n'est pas bien meilleure, alors que la complexité de celle-ci présageait un gain net de temps. Malheureusement, le processeur utilisé n'est pas massivement parallèle. En conséquence, le gain prodigué par la parallélisation des comparaisons est négligeable. Cela nous permet d'observer que le tri pair-impair n'est pas si intéressant sur un processeur "classique".

Le graphique général des tris par comparaisons laisse à supposer que le tri rapide est le plus performants de tous ceux étudiés. Les limites matérielles de la machine de tests nous aurait induits en erreur quant à cette idée. Nous voyons que sur des nombres importants de valeurs à trier, le tri rapide est dépassé largement par ses deux "concurrents". C'est donc dans les petits nombres de valeurs que celui-ci est performant. Le tri fusion apparaît comme le plus performant, mais gardons à l'esprit qu'il utilise beaucoup de mémoire externe. Le tri par tas se pose comme alternative, moins rapide que le tri fusion, mais moins coûteux en mémoire.

Concernant les tris sans comparaisons, il est assez clair que l'intervalle des valeurs à trier influe énormément sur les performances de ces algorithmes. Leur vitesse peut drastiquement changer en fonction de cette intervalle. Le tri comptage est un bon exemple, celui-ci étant vraiment lent sur de gros intervalles alors qu'il bat tous les autres question vitesse sur des petits. De plus, il utilise la mémoire externe proportionnellement à cet intervalle. Mais cet algorithme présente une courbe curieuse... croissant très vite, puis se stabilisant peu à peu. Si la machine de tests avait été plus puissante, nous aurions peut-être pu voir un rapprochement de toutes les courbes. Le tri par base est légèrement plus lent sur les grands intervalles car la taille des nombres en terme de chiffres est forcément plus important que sur les petits. Le tri par paquets peut nous surprendre plus, un intervalle plus grand le rend plus rapide. Étonnant à premier abord, n'est-ce pas ? Et pourtant, c'est facilement

compréhensible. Cet algorithme se base sur la répartition des valeurs. Or, si elles sont générés aléatoirement sur de grands intervalles, elles sont généralement mieux réparties. Il faut se rappeler que le meilleur cas de ce tri est quand chaque paquet ne contient qu'une seule valeur. En somme, moins il y a de valeurs par paquets, plus l'algorithme est rapide, donc une répartition homogène des valeurs amène à de meilleures performances.

3.5 Conclusion

La conclusion que nous tirons de cette étude est qu'il n'y a pas d'algorithme parfait... un algorithme performant sur un point "pêche" généralement sur un autre, qui sera le point fort d'un algorithme différent. Le choix d'un algorithme de tri dépend de la machine cible, et des données à trier. Une machine avec très peu de mémoire ne pourra malheureusement pas se voir gratifier d'algorithmes comme le tri fusion, le tri comptage, etc... Mais ce n'est pas si évident. Si la machine cible ne doit que trier des petites valeurs, le tri comptage pourrait très bien faire l'affaire, car il n'utilisera que très peu de mémoire, tout en ayant des performances très acceptables. Nous voyons donc quels sont les points importants à prendre en compte lorsque l'on réfléchit à l'utilisation d'un algorithme de tri, et lorsque nous essayons d'en créer un : la vitesse, le coût mémoire et les intervalles de données.

Chapitre 4

Algorithmes

Pour ceux qui sont curieux de connaître les algorithmes que nous avons utilisés, les voici... ils sont assez généralistes, mais nous les avons codé nous-même, il peuvent de ce fait être complètement différent de ceux que l'on trouve dans les livres ou sites internet.

4.1 Tri à bulles

Algorithm 1 Algorithme du tri à bulles

Entrée(s): $tab : \text{tableau}[1 \dots n]$, $n \in \mathbb{N}$ **Sortie(s):** $tab : \text{tableau trié}$

```
1: for  $i \leftarrow 1$  to  $n - 1$  do
2:   for  $j \leftarrow 1$  to  $j < n - i$  do
3:     if  $tab[j] > tab[j + 1]$  then
4:       Echanger( $tab[j]$ ,  $tab[j + 1]$ )
5:     end if
6:   end for
7: end for
```

4.2 Tri sélection

Algorithm 2 Algorithme du tri sélection

Entrée(s): $tab : \text{tableau}[1 \dots n]$, $n \in \mathbb{N}$ **Sortie(s):** $tab : \text{tableau trié}$

```
1: for  $i \leftarrow 1$  to  $n - 1$  do
2:    $jmin \leftarrow i$ 
3:   for  $j \leftarrow i + 1$  to  $j \leq n$  do
4:     if  $tab[jmin] > tab[j]$  then
5:        $jmin \leftarrow j$ 
6:     end if
7:   end for
8:   Echanger( $tab[jmin]$ ,  $tab[i]$ )
9: end for
```

4.3 Tri insertion

Algorithm 3 Algorithme du tri insertion

Entrée(s): $tab : tableau[1 \dots n]$, $n \in \mathbb{N}$

Sortie(s): tab : tableau trié

```
1: for  $i \leftarrow 1$  to  $n$  do
2:    $tmp \leftarrow tab[i]$ 
3:    $j \leftarrow i$ 
4:   while  $j > 1$  and  $tab[j-1] > tmp$  do
5:      $tab[j] \leftarrow tab[j-1]$ 
6:      $j \leftarrow j-1$ 
7:   end while
8:    $tab[j] \leftarrow tmp$ 
9: end for
```

4.4 Tri rapide

Algorithm 4 Partitionnement d'un tableau

Fonction Partition(tab , $gauche$, $droite$)

Entrée(s): $tab : tableau[gauche \dots droite]$, $(gauche, droite) \in \mathbb{N}^2$

Sortie(s): $pivot$: position du pivot dans le tableau

```
1:  $i \leftarrow gauche$ 
2:  $j \leftarrow droite$ 
3:  $pivot \leftarrow tab[droite]$  {On choisit le dernier élément du tableau comme pivot}
4: while  $i < j$  do
5:   while  $tab[i] < pivot$  and  $i < droite$  do
6:      $i \leftarrow i + 1$ 
7:   end while
8:   while  $tab[j] > pivot$  and  $i < j$  do
9:      $j \leftarrow j - 1$ 
10:  end while
11:  Echanger( $tab[i]$ ,  $tab[j]$ )
12: end while
13: if  $tab[j] > tab[droite]$  then
14:   Echanger( $tab[j]$ ,  $tab[droite]$ )
15: end if
16: return  $pivot$ 
```

Algorithm 5 Algorithme du tri rapide

Procédure TriRapide(tab , $gauche$, $droite$)

Entrée(s): $tab : tableau[gauche \dots droite]$, $(gauche, droite) \in \mathbb{N}^2$

Sortie(s): tab : tableau trié de $gauche$ à $droite$

```
1: if  $gauche < droite$  then
2:    $pivot \leftarrow$  Partition( $tab$ ,  $gauche$ ,  $droite$ )
3:   TriRapide( $tab$ ,  $gauche$ ,  $pivot - 1$ )
4:   TriRapide( $tab$ ,  $pivot + 1$ ,  $droite$ )
5: end if
```

4.5 Tri fusion

Algorithm 6 Fusion de deux partie du tableau

Procédure Fusion(*tab*, *gauche*, *milieu*, *droite*)

Entrée(s): *tab* : tableau[*gauche*...*droite*], (*gauche*, *milieu*, *droite*) $\in \mathbb{N}^3$

Sortie(s): *tab* : tableau fusionné et trié

```
1: aux  $\leftarrow$  tableau[gauche...droite]
2: longueur  $\leftarrow$  (droite - gauche) + 1
3: i  $\leftarrow$  gauche
4: j  $\leftarrow$  milieu + 1
5: k  $\leftarrow$  1
6: while k  $\leq$  longueur do
7:   if i  $\leq$  milieu and j  $\leq$  droite then
8:     if tab[i] < tab[j] then
9:       aux[k]  $\leftarrow$  tab[i]
10:      i  $\leftarrow$  i + 1
11:     else
12:       aux[k]  $\leftarrow$  tab[j]
13:       j  $\leftarrow$  j + 1
14:     end if
15:   end if
16:   k  $\leftarrow$  k + 1
17: end while
18: for i  $\leftarrow$  1 to longueur do
19:   tab[gauche + i]  $\leftarrow$  aux[i]
20: end for
```

Algorithm 7 Algorithme du tri fusion

Procédure TriFusion(*tab*, *gauche*, *droite*)

Entrée(s): *tab* : tableau[*gauche*...*droite*], (*gauche*, *droite*) $\in \mathbb{N}^2$

Sortie(s): *tab* : tableau trié

```
1: if gauche < droite then
2:   milieu  $\leftarrow$  (gauche + droite)/2
3:   TriFusion(tab, left, milieu)
4:   TriFusion(tab, milieu + 1, right)
5:   Fusion(tab, gauche, milieu, droite)
6: end if
```

4.6 Tri par tas

Algorithm 8 Création du tas

Procédure *Tamiser*(*arbre*, *noeud*, *n*)

Entrée(s): *arbre* : *tableau*[1 ... *n*], (*noeud*, *n*) ∈ ℕ²

Sortie(s): *arbre* : *tas*

```
1: k ← noeud
2: j ← 2 * k
3: while j < n do
4:   if j < n and arbre[j] < arbre[j + 1] then
5:     j ← j + 1
6:   end if
7:   if arbre[k] < arbre[j] then
8:     Echanger (arbre[k], arbre[j])
9:     k ← j
10:    j ← 2 * k
11:  else
12:    j ← n {Sortie de boucle}
13:  end if
14: end while
```

Algorithm 9 Algorithme du tri par tas

Entrée(s): *arbre* : *tableau*[1 ... *n*], *n* ∈ ℕ

Sortie(s): *tab* : *tableau trié*

```
1: for i ← n/2 to 1 step −1 do
2:   Tamiser (arbre, i, n)
3: end for
4: for i ← n to 2 step −1 do
5:   Echanger(arbre[i], arbre[1])
6:   Tamiser (arbre, 1, i − 1)
7: end for
```

4.7 Tri par base

Algorithm 10 Algorithme du tri par base

Entrée(s): $tab : \text{tableau}[1 \dots n]$, $n \in \mathbb{N}$

Sortie(s): $tab : \text{tableau trié}$

```
1:  $files \leftarrow File[1 \dots 10]$ 
2:  $sommeCle \leftarrow 1$ 
3: while  $sommeCle \neq 0$  do
4:    $sommeCle \leftarrow 0$ 
5:   for  $i \leftarrow 1$  to  $n$  do
6:      $cle \leftarrow$  chiffre au rang  $i$  {Extraction de chaque chiffre}
7:      $Enfiler(files[cle], tab[i])$ 
8:      $sommeCle \leftarrow sommeCle + cle$ 
9:   end for
10:  if  $sommeCle \neq 0$  then
11:     $k \leftarrow 1$ 
12:    for  $i \leftarrow 10$  to  $1$  step  $-1$  do
13:      for  $j \leftarrow 1$  to  $Longueur(files[i])$  do
14:        if  $Tete(files[i]) < 0$  then
15:           $Défiler(tab[k], files[i])$ 
16:           $k \leftarrow k + 1$ 
17:        end if
18:      end for
19:    end for
20:    for  $i \leftarrow 1$  to  $10$  do
21:      for  $j \leftarrow 1$  to  $Longueur(files[i])$  do
22:        if  $Tete(files[i]) \geq 0$  then
23:           $Défiler(tab[k], files[i])$ 
24:           $k \leftarrow k + 1$ 
25:        end if
26:      end for
27:    end for
28:  end if
29: end while
```

4.8 Tri comptage

Algorithm 11 Algorithme du tri comptage

Entrée(s): $tab : \text{tableau}[1 \dots n]$, $n \in \mathbb{N}$, $(min, max) \in \mathbb{N}^2$

Sortie(s): $tab : \text{tableau trié}$

```
1:  $longueur \leftarrow (max - min) + 1$ 
2:  $comptage \leftarrow \text{tableau}[1 \dots longueur]$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:    $comptage[(tab[i] - min) + 1] \leftarrow comptage[(tab[i] - min) + 1] + 1$ 
5: end for
6:  $k \leftarrow 1$ 
7: for  $i \leftarrow 1$  to  $longueur$  do
8:   for  $j \leftarrow 1$  to  $comptage[i]$  do
9:      $tab[k] \leftarrow (i + min) - 1$ 
10:     $k \leftarrow k + 1$ 
11:   end for
12: end for
```

4.9 Tri par paquets

Algorithm 12 Algorithme du tri par paquets

Entrée(s): $tab : tableau[1 \dots n]$, $n \in \mathbb{N}$, $(a, b) \in \mathbb{N}^2$

Sortie(s): $tab : tableau$ trié

```
1:  $paquets \leftarrow Liste[1 \dots n]$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $k \leftarrow n(tab[i] - a)/(b - a)$  {Détermine à quel sous-intervalle appartient la valeur au rang  $i$ }
4:   AjouterDansListe( $paquets[k]$ ,  $tab[i]$ );
5: end for
6:  $k \leftarrow 1$ 
7: for  $i \leftarrow 1$  to  $n$  do
8:   for  $j \leftarrow 1$  to Longueur( $paquets[i]$ ) do
9:      $tab[k] \leftarrow paquets[i][j]$ 
10:     $k \leftarrow k + 1$ 
11:   end for
12: end for
```

4.10 Tri pair-impair

Algorithm 13 Algorithme du tri pair-impair

Entrée(s): $tab : tableau[1 \dots n]$, $n \in \mathbb{N}$

Sortie(s): $tab : tableau$ trié

```
1: for  $i \leftarrow 1$  to  $n - 1$  do
2:   if  $i$  est pair then
3:     for  $j \leftarrow 2$  to  $j < n - 1$  do
4:       if  $tab[j] > tab[j + 1]$  then
5:         Echanger( $tab[j]$ ,  $tab[j + 1]$ )
6:       end if
7:     end for
8:   else
9:     for  $j \leftarrow 1$  to  $j < n - 1$  do
10:      if  $tab[j] > tab[j + 1]$  then
11:        Echanger( $tab[j]$ ,  $tab[j + 1]$ )
12:      end if
13:    end for
14:   end if
15: end for
```

Chapitre 5

Codes sources

Bien que cela soit superflu, nous avons pensé qu’il était de bon ton de donner nos implémentations des algorithmes en C dont nous nous sommes servis pour les tests de performances. Puisqu’elles sont basées sur les algorithmes précédents, elles peuvent être complètement différentes des implémentations “classiques” que l’on trouve un peu partout sur l’internet.

```
void bubbleSort(int *tab, int n)
{
    int i, j, tmp;

    for (i = 0 ; i < n - 1 ; i++) {
        for (j = 0 ; j < n - (i + 1) ; j++) {
            if (tab[j] > tab[j + 1]) {
                tmp = tab[j];
                tab[j] = tab[j + 1];
                tab[j + 1] = tmp;
            }
        }
    }
}
```

Listing 5.1 – Tri à bulles

```
void selectionSort(int *tab, int n)
{
    int i, j, jmin;

    for (i = 0 ; i < n - 1 ; i++) {
        jmin = i;

        for (j = i + 1 ; j < n ; j++)
            if (tab[jmin] > tab[j]) jmin = j;

        swap(&tab[jmin], &tab[i]);
    }
}
```

Listing 5.2 – Tri sélection

```

void insertionSort(int *tab, int n)
{
    int i, j, tmp;

    for (i = 1 ; i < n ; i++) {
        tmp = tab[i];

        for (j = i ; j > 0 && tab[j - 1] > tmp ; j--)
            tab[j] = tab[j - 1];

        tab[j] = tmp;
    }
}

```

Listing 5.3 – Tri insertion

```

void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int partition(int *tab, int left, int right)
{
    int i = left, j = right - 1;
    int pivot = tab[right];

    while (i < j) {
        while (tab[i] < pivot && i < right) i++;
        while (tab[j] >= pivot && i < j) j--;

        swap(&tab[i], &tab[j]);
    }

    if (tab[j] > tab[right]) swap(&tab[j], &tab[right]);

    return j;
}

void quickSort(int *tab, int left, int right)
{
    int pivot;

    if (left >= right) return;

    pivot = partition(tab, left, right);
    quickSort(tab, left, pivot - 1);
    quickSort(tab, pivot + 1, right);
}

```

Listing 5.4 – Tri rapide

```

void merge(int *tab, int left, int middle, int right)
{
    int length = (right - left) + 1;
    int *aux = malloc(length * sizeof(int));
    int i, j, k;

    for (i = left, j = middle + 1, k = 0 ; k < length ; k++) {
        if (i <= middle && j <= right)
            aux[k] = (tab[i] < tab[j]) ? tab[i++] : tab[j++];
        else {
            if (i <= middle) aux[k] = tab[i++];
            else aux[k] = tab[j++];
        }
    }

    for (i = 0 ; i < length ; i++) tab[left + i] = aux[i];
    free(aux);
}

void mergeSort(int *tab, int left, int right)
{
    int middle = (right + left) / 2;

    if (left >= right) return;

    mergeSort(tab, left, middle);
    mergeSort(tab, middle + 1, right);
    merge(tab, left, middle, right);
}

```

Listing 5.5 – Tri fusion

```

void heapSwap(int* tree, int node1, int node2)
{
    int temp = tree[node1];
    tree[node1] = tree[node2];
    tree[node2] = temp;
}

void sift(int* tree, int node, int n)
{
    int k = node;
    int j = 2 * k;

    while (j < n) {
        if (tree[j] < tree[j + 1]) j++;
        if (tree[k] < tree[j]) {
            heapSwap(tree, k, j);
            k = j;
            j = 2 * k;
        }
        else return;
    }
}

void heapSort(int* tree, int n)
{
    int i;

    for (i = n / 2; i >= 0; i--) sift(tree, i, n);
    for (i = n - 1; i >= 1; i--) {
        heapSwap(tree, i, 0);
        sift(tree, 0, i - 1);
    }
}

```

Listing 5.6 – Tri par tas

```

void radixSort(int *tab, int n)
{
    int (*stack)[10] = malloc(n * sizeof(int [10]));
    int topStack[10];
    int keySum = 1, base = 1, key;
    int i, j, k;

    while (keySum) {
        keySum = 0;
        memset(topStack, 0, 10 * sizeof(int));

        for (i = 0 ; i < n ; i++) {
            key = (abs(tab[i]) / base) % 10;
            stack[topStack[key]++][key] = tab[i];
            keySum += key;
        }

        if (keySum != 0) {
            for (i = 9, k = 0 ; i >= 0 ; i--)
                for (j = 0 ; j < topStack[i] ; j++)
                    if (stack[j][i] < 0) tab[k++] = stack[j][i];

            for (i = 0 ; i < 10 ; i++)
                for (j = 0 ; j < topStack[i] ; j++)
                    if (stack[j][i] >= 0) tab[k++] = stack[j][i];

            base *= 10;
        }
    }

    free(stack);
}

```

Listing 5.7 – Tri par base

```

void countingSort(int *tab, int n, int boundInf, int boundSup)
{
    int i, j, k;
    int length = (boundSup - boundInf) + 1;
    int *cpt = calloc(length, sizeof(int));

    for (i = 0 ; i < n ; i++)
        if (tab[i] >= boundInf && tab[i] <= boundSup)
            cpt[tab[i] - boundInf]++;

    for (i = 0, k = 0 ; i < length ; i++)
        for (j = 0 ; j < cpt[i] ; j++) tab[k++] = i + boundInf;

    free(cpt);
}

```

Listing 5.8 – Tri comptage

```

void bucketSort(int *tab, int n, int a, int b)
{
    int *aux;
    int i, j, k;

    if (a >= b) return;

    List **buckets = malloc(n * sizeof(List *));
    for (i = 0 ; i < n ; i++) buckets[i] = List_create();

    for (i = 0 ; i < n ; i++) {
        k = n * (tab[i] - a) / (b - a);
        List_add(buckets[k], tab[i]);
    }

    for (i = 0, k = 0 ; i < n ; i++) {
        aux = List_toArray(buckets[i]);
        quickSort(aux, 0, buckets[i]->length - 1);

        for (j = 0 ; j < buckets[i]->length ; j++) tab[k++] = aux[j];
        List_free(buckets[i]);
        free(aux);
    }

    free(buckets);
}

```

Listing 5.9 – Tri par paquets (avec tri rapide)

```

void oddevenSort(int *tab, int n)
{
    int i, j;

    for (i = 0 ; i < n - 1 ; i++) {
        if ((i & 1) == 0)
            for (j = 0 ; j < n - 1 ; j += 2) {
                if (tab[j] > tab[j + 1]) swap(&tab[j], &tab[j + 1]);
            }

        else
            for (j = 1 ; j < n - 1 ; j += 2) {
                if (tab[j] > tab[j + 1]) swap(&tab[j], &tab[j + 1]);
            }
    }
}

```

Listing 5.10 – Tri pair-impair

```

typedef struct Node
{
    int value;
    struct Node *next;
} Node;

typedef struct List
{
    int length;
    struct Node *first;
    struct Node *last;
} List;

List* List_create()
{
    List *l = malloc(sizeof(List));

    l->length = 0;
    l->first = NULL;
    l->last = NULL;
    return l;
}

void List_add(List *l, int value)
{
    Node *new = malloc(sizeof(Node));

    new->value = value;
    new->next = NULL;

    if (l->first == NULL) l->first = new;
    else l->last->next = new;

    l->last = new;
    l->length++;
}

int* List_toArray(List *l)
{
    Node *cur;
    int *tab = malloc(l->length * sizeof(int));
    int i;

    for (cur = l->first, i = 0 ; cur != NULL ; cur = cur->next, i++)
        tab[i] = cur->value;

    return tab;
}

void List_free(List *l)
{
    Node *cur = l->first, *tmp;

    while (cur != NULL) {
        tmp = cur;
        cur = cur->next;
        free(tmp);
    }

    free(l);
}

```

Listing 5.11 – Implémentation des listes

Bibliographie

- [1] Donald Edgar Knuth. *The Art of Computer Programming, Sorting and Searching*, volume 3. 1998.
- [2] R. Rivest C. Stein T. Cormen, C. Leiserson. *Introduction à l'algorithmique*. 2001.